Evaluating Cross-Architecture Robustness in Adversarial Neural Cryptography

CS 180H Research Proposal | Department of Computer Science | San José State University

Kunj Shah | Student ID: 016970226 | Faculty advisor: Professor Mark Stamp
kunj.shah@sjsu.edu | kunjcr2.github.io

1. Project summary

Adversarial neural cryptography asks whether neural networks can learn keyed communication through a contest among Alice (the sender), Bob (the receiver), and Eve (an eavesdropper) [1]. Alice turns a binary message and shared key into ciphertext; Bob uses the key to recover the message, while Eve tries to recover it from the ciphertext alone. Alice and Bob may appear successful against the Eve they trained with, while a different attacker can still recover information. This project will compare Eve architectures, including a state-space model, and test whether attacker size and training time change the results. The goal is a reproducible evaluation of learned communication, not a replacement for established cryptography or a claim of formal secrecy.

2. Research question and motivation

The central question is: *How well does an Alice–Bob pair trained with one kind of Eve resist fresh attackers built with other architectures?* The project will compare CNN, MLP, bidirectional LSTM, Transformer, and state-space Eves. It will also examine whether attacker size and training time affect attack success. A fresh Eve is important because a weak in-training attacker does not show that other attackers will also fail. Prior work has retrained Eves after freezing Alice and Bob [1] and explored stronger attackers and security goals [2, 3, 4]. This project will compare attackers against the same frozen Alice–Bob pair, keeping the communication system fixed during each comparison.

3. Preliminary work

I have built a PyTorch prototype of the Alice–Bob–Eve setup and run initial comparisons among CNN, MLP, bidirectional LSTM, and Transformer attackers. Early runs suggest that a larger attacker trained for longer can sometimes recover more of the message, while some Alice–Bob runs fail to communicate reliably. These results are preliminary: they use few repeated runs and do not establish a general ranking or a security guarantee. They motivate the proposed replication, a communication-quality check, and the addition of a state-space Eve. Existing notebooks and experiment notes provide a starting point for the semester.

4. Proposed method

1. **Build and check the experiment.** Use randomly generated 32-bit messages and keys and the existing Alice–Bob setup. Train communication pairs against CNN, MLP, bidirectional LSTM, Transformer, and state-space Eves. Keep the data splits and training setup consistent, and save seeds and model settings. Include simple controls to verify that the attack code can detect recoverable messages.
2. **Check communication first.** Test Bob on new message–key pairs and record both bit-level and complete-message accuracy. Require at least 96% bit accuracy on validation and test data before interpreting Eve's results. Record how often each training setup fails this check.
3. **Freeze and attack.** Freeze each pair that passes the communication check. Train fresh ciphertext-only CNN, MLP, bidirectional LSTM, Transformer, and state-space attackers. Select settings using validation data and use a separate test set for final results. The main comparison will use one manageable attacker size and a fixed training budget. A smaller follow-up comparison will test a smaller versus larger attacker and shorter versus longer training.
4. **Repeat and analyze.** Aim for three independently trained Alice–Bob pairs per training-Eve architecture and two fresh attacker starts per main comparison. Report results across pair seeds and attacker starts separately. Failed communication runs will be reported, but not treated as evidence of privacy. Revisit preliminary anomalies by checking saved samples, checkpoints, and fresh attackers.

The primary outcome is fresh Eve's accuracy on held-out message bits; 50% is chance for balanced independent bits. Other outcomes are complete-message recovery, bit errors, Bob's accuracy, and how often training produces a reliable pair. The analysis will compare Eve architectures on the same frozen pair. The focused follow-up will explore attacker size and training time. Since architectures differ in more than size, the report will describe the models and training budgets rather than assume equal parameter counts mean equal ability.

5. Scope, resources, and deliverables

The project will build on the existing PyTorch prototype, notebooks, and experiment notes. University or cloud GPU access will support repeated runs. If compute is limited, the size-and-time follow-up will be reduced before the main architecture comparison. Messages and keys will be randomly generated; no private dataset is needed. Seeds, settings, data splits, and results will be saved so the experiment can be repeated.

The deliverables are: (1) a documented experiment and attack codebase; (2) a replicated comparison of Eve architectures, including state-space models; (3) a focused comparison of attacker size and training time; (4) an explanation of failed communication runs and notable anomalies; and (5) a research report and presentation with limitations.

5.1 Sixteen-week plan

The schedule follows the 16-week Spring 2027 span, including spring recess. Weeks are project targets and may shift with compute availability.

Week	Plan
1	Review the proposal and existing prototype; settle the experiment questions and success measures.
2	Reproduce the current Alice–Bob–Eve setup and record a baseline run.
3	Add the communication check, held-out data, and simple attack controls.
4	Implement and check the CNN and MLP Eve variants.
5	Add the bidirectional LSTM and Transformer Eve variants.
6	Add a state-space Eve and check that it trains on the same task.
7	Run a small comparison across training-Eve architectures; fix reliability or runtime issues.
8	Freeze the protocol, seeds, and primary experiment settings.
9	Spring recess / buffer: review early results and catch up on implementation if needed.
10	Train the main Alice–Bob pairs across Eve architectures.
11	Apply the communication gate; repeat or document failed runs.
12	Train fresh attackers of each architecture against the accepted frozen pairs.
13	Complete the main repeated runs and summarize architecture comparisons.
14	Run the smaller-versus-larger and shorter-versus-longer follow-up on selected conditions.
15	Analyze results, make figures, and draft the report and presentation.
16	Revise the report, document code and limitations, and present the findings.

The main architecture comparison takes priority over the size-and-time follow-up. Optional extensions such as ensemble attackers or chosen-plaintext attacks will be attempted only if the core study and report are on schedule.

6. Expected contribution and limits

The expected contribution is a controlled, reusable way to test whether a learned communication scheme has merely adapted to its training attacker. The project may find cross-architecture leakage, architecture-specific capacity effects, or more robust behavior under some conditions. Any of these outcomes would clarify how neural cryptography experiments should be evaluated. Failure of the tested Eves to recover plaintext would establish only empirical resistance to those Eves and budgets. It would not prove semantic security or resistance to other learned or analytical attacks. The broader literature includes topology changes, longer messages, asymmetric settings, multi-attacker systems, and a recent survey of open evaluation challenges [5, 6, 7, 9, 10]; those topics inform the discussion but are outside the controlled core study.

References

[1] M. Abadi and D. G. Andersen, "Learning to Protect Communications with Adversarial Neural Cryptography," 2016. https://arxiv.org/abs/1610.06918.

[2] M. Coutinho et al., "Learning Perfectly Secure Cryptography to Protect Communications with Adversarial Neural Cryptography," *Sensors*, 2018.

[3] L. Zhou et al., "Security Analysis and New Models on the Intelligent Symmetric Key Encryption," *Computers & Security*, 2019.

[4] Z. Li et al., "Information Encryption Communication System Based on the Adversarial Networks Foundation," *Neurocomputing*, 2020.

[5] Y. Zhu, D. V. Vargas, and K. Sakurai, "Neural Cryptography Based on the Topology Evolving Neural Networks," *CANDAR Workshops*, 2018.

[6] X. Duan et al., "Optimization of Encrypted Communication Length Based on Generative Adversarial Network," *IEEE BDAI*, 2021.

[7] I. Meraouche et al., "Learning Asymmetric Encryption Using Adversarial Neural Networks," *Engineering Applications of Artificial Intelligence*, 2023.

[8] J. H. Tyler, M. K. M. Fadul, and D. R. Reising, "Transformers—Messages in Disguise," *Machine Learning with Applications*, 2025. https://doi.org/10.1016/j.mlwa.2025.100669.

[9] Y. Wang et al., "AdvOT: Oblivious Transfer Based on Generative Adversarial Networks Against Multiple Attackers," *Neurocomputing*, 2025.

[10] Y. Zhong, J. Gu, and X. Yu, "Neural Cryptography: Synergy and Conflict Between Neural Networks and Cryptographic Systems," *Neurocomputing*, 2026. https://doi.org/10.1016/j.neucom.2026.133084.